LAPORAN PRAKTIKUM
SISTEM OPERASI

Modul 06 - Proses dan Manajemen Proses



NAMA	: Rosyid Al Ansorri
NIM	: L0125065
KELAS	: A
HARI, TANGGAL	: Jumat, 16 Oktober 2026
WAKTU	: 09:10 - selesai
DOSEN	: Ery Permana Yudha, S.Kom., M.Kom.
ASISTEN	: Benedhictus Kevin Doni (L0124006)
	Daffa Arrivo Aryanto (L0124093)

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

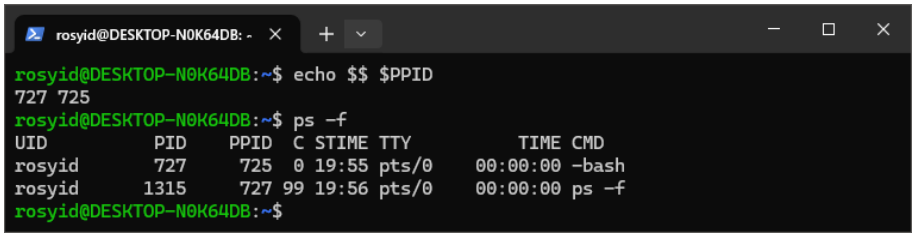
2026

BAB I
LANGKAH DAN PEMBAHASAN

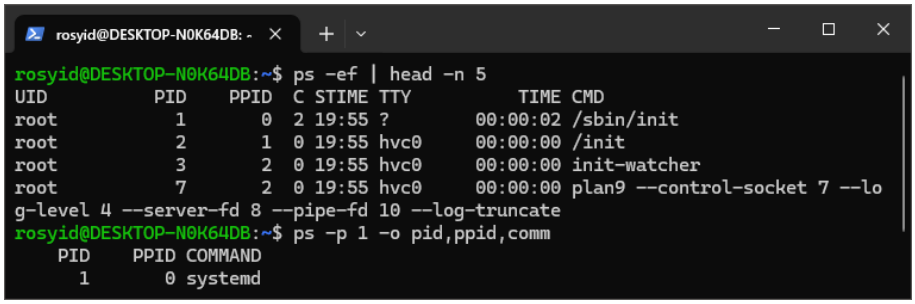
KEGIATAN 1

A. Dokumentasi Kegiatan 1

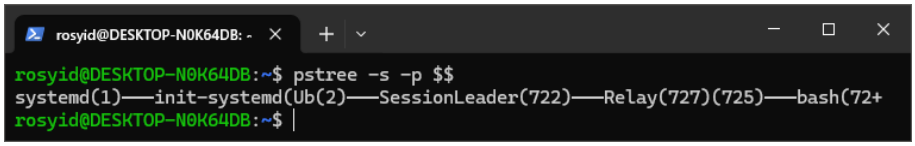
1. Menampilkan PID dan PPID shell serta detail proses dengan `ps -f`



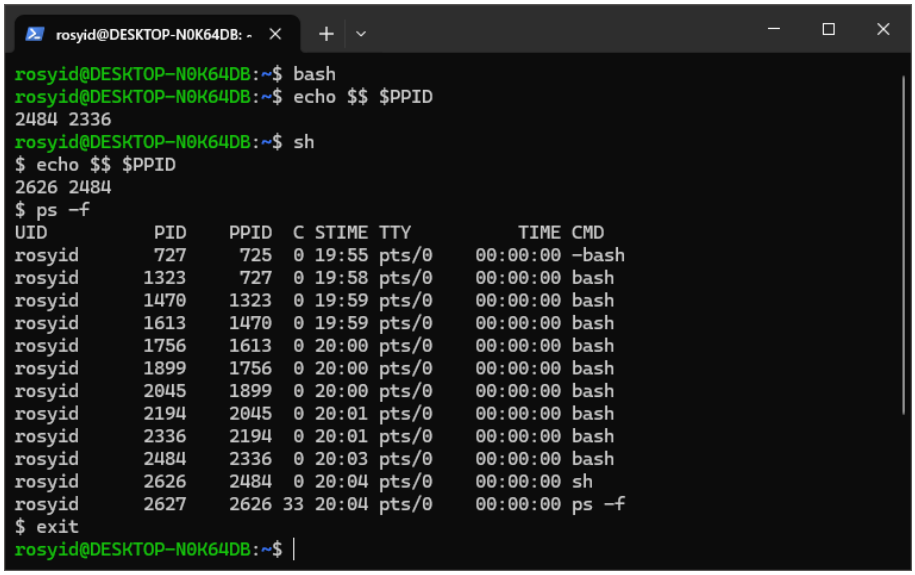
2. Menampilkan proses awal sistem dan mencari proses PID 1



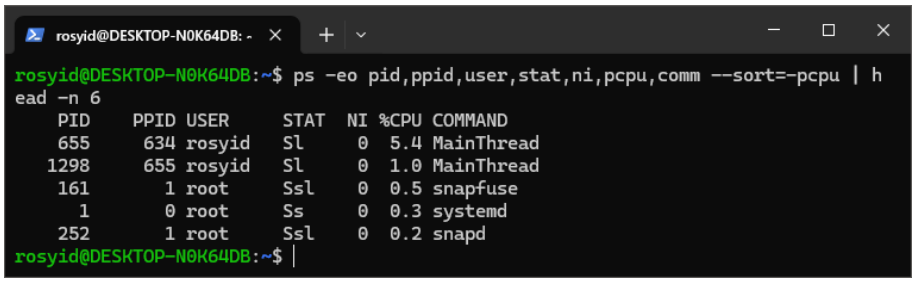
3. Menampilkan silsilah shell sampai PID 1 dengan `pstree`



4. Membuat shell bertingkat (`bash` dan `sh`) dan mencatat PID/PPID



5. Menampilkan proses dengan penggunaan CPU terbesar



6. Menampilkan proses daemon



7. Catat hasil pengamatan

Aspek Pengamatan	Output Terminal
PID dan PPID shell awal	PID 727, PPID 725 (dari echo $$ $PPID)
Nama proses PID 1	systemd (ps -p 1 -o pid,ppid,comm: PID 1, PPID 0)
Silsilah shell sampai PID 1 (hasil `pstree`)	systemd(1)—init-systemd(Ub(2)—SessionLeader(722)—Relay(727)(725)—bash(72+
PID dan PPID `bash` (tingkat 1) dan `sh` (tingkat 2)	bash: PID 2484, PPID 2336 sh: PID 2626, PPID 2484
PID dan COMMAND proses dengan %CPU tertinggi	PID 655, COMMAND MainThread, %CPU 5.4
Dua contoh proses daemon (PID dan nama)	PID 252 snapd PID 322 dockerd

B. Jawaban Analisis Kegiatan 1

1. PPID sh (tingkat 2) sama dengan PID bash (tingkat 1) karena sh dijalankan sebagai proses anak dari bash. Setiap perintah atau shell baru dibuat dengan fork dari proses pemanggilnya, sehingga proses pemanggil menjadi parent dan proses baru menjadi child.

2. PID 1 adalah proses induk dari seluruh proses lain dan bertugas menjalankan layanan sistem saat boot, serta mengadopsi proses yatim (orphan). Pada Linux modern PID 1 adalah systemd, yang menjalankan layanan secara paralel dan dikelola melalui unit file dengan perintah systemctl. Pada sistem lama, PID 1 adalah init yang dikonfigurasi lewat /etc/inittab dan bekerja dengan runlevel secara berurutan.
3. Nama proses daemon diberi akhiran d (daemon) sebagai konvensi untuk menandai proses layanan. Ciri umumnya: berjalan di background tanpa terminal kontrol, terus hidup menunggu permintaan, sering dijalankan saat boot, dan umumnya memiliki PPID 1. Contoh yang teramati: snapd dan dockerd dengan PPID 1.

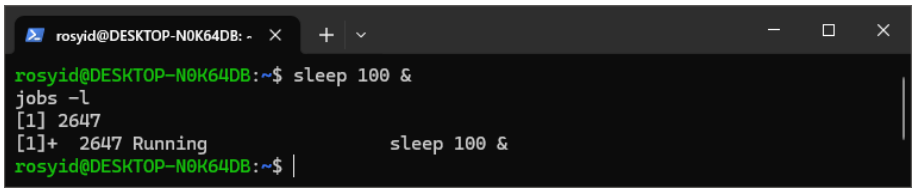
KEGIATAN 2

A. Dokumentasi Kegiatan 2

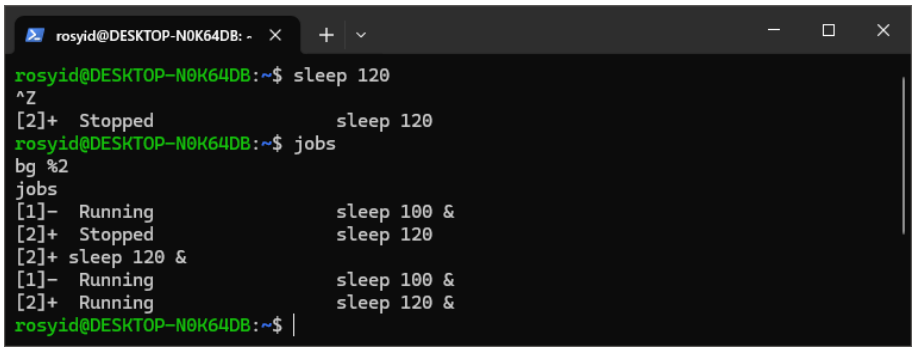
1. Menjalankan `sleep 5` di foreground



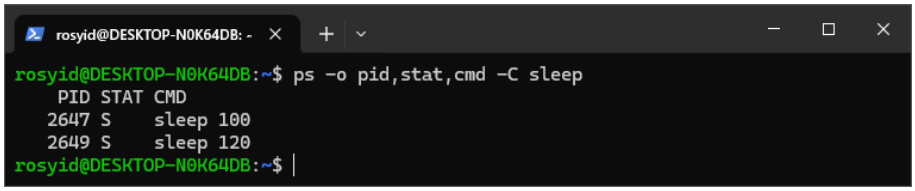
2. Menjalankan `sleep 100 &` di background dan melihat `jobs -l`



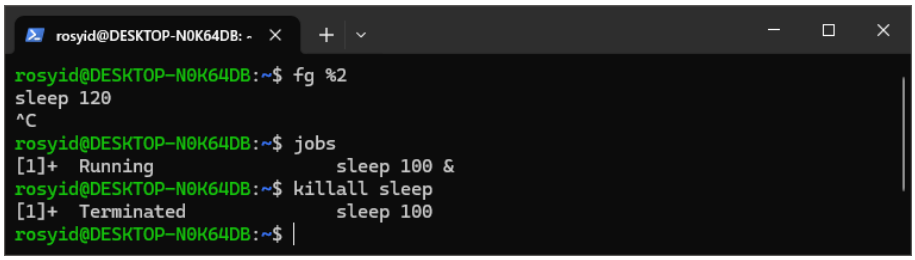
3. Menghentikan sementara job dengan Ctrl+Z, lalu melanjutkan dengan `bg`



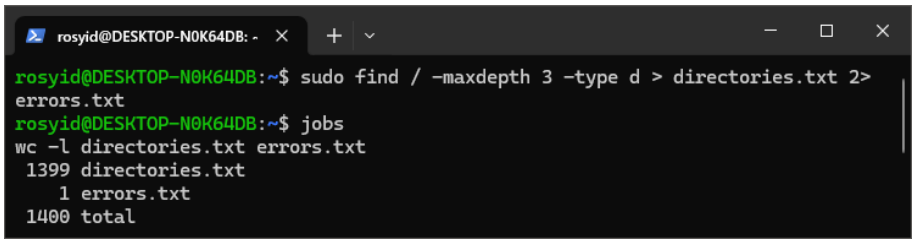
4. Memeriksa status proses `sleep` dengan `ps -o pid,stat,cmd -C sleep`



5. Memindahkan job ke foreground (`fg`), menginterupsi dengan Ctrl+C, lalu membersihkan dengan `killall sleep`



6. Menjalankan `find` di background dengan redirect > dan 2>



7. Catat hasil pengamatan

Aspek Pengamatan	Output Terminal
Perilaku prompt saat `sleep 5` di foreground	Prompt kembali setelah sleep 5 selesai, tanpa keluaran tambahan
Isi `jobs -l` setelah `sleep 100 &`	[1] 2647 [1]+ 2647 Running sleep 100 &
STAT proses `sleep 120` setelah Ctrl+Z (sebelum `bg`)	[2]+ Stopped sleep 120 (status T pada jobs)
STAT proses `sleep 120` setelah `bg`	2649 S sleep 120 [2]+ Running sleep 120 & setelah bg
Jumlah baris `directories.txt` dan `errors.txt`	directories.txt: 1399 baris errors.txt: 1 baris total: 1400

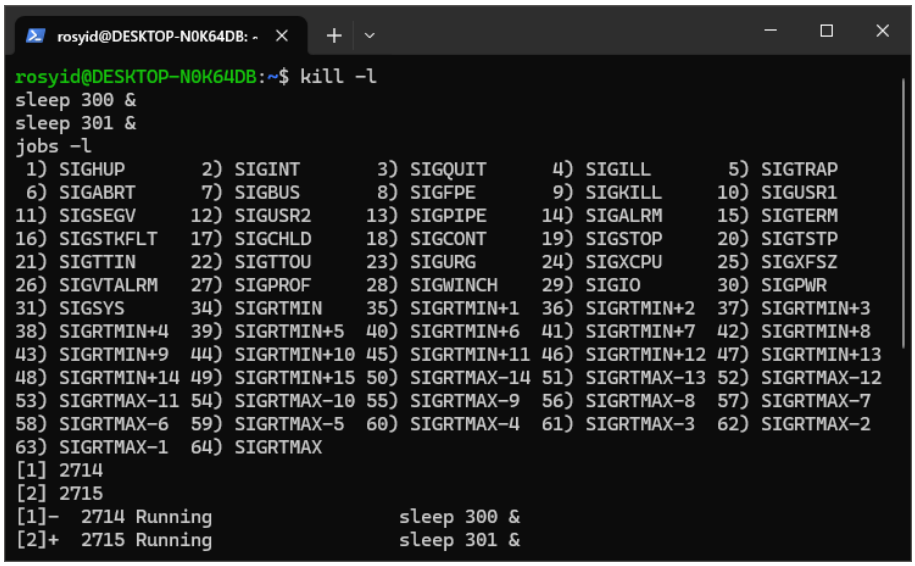
B. Jawaban Analisis Kegiatan 2

1. Ctrl+Z mengirim SIGTSTP dan menghentikan sementara job (status T, Stopped) sehingga dapat dilanjutkan kembali dengan bg atau fg. Ctrl+C mengirim SIGINT dan mengakhiri job secara permanen sehingga proses tidak lagi muncul pada jobs atau ps. Pada praktikum, sleep 120 yang dihentikan dengan Ctrl+Z berstatus T, sedangkan setelah bg statusnya kembali S (sleeping).
2. Job di background tidak menerima input dari keyboard terminal, sehingga Ctrl+C (SIGINT yang dikirim ke grup proses foreground) tidak mencapainya. Cara menghentikannya adalah membawa job ke foreground dengan fg %n lalu menekan Ctrl+C, atau mengirim sinyal langsung dengan kill %n atau kill PID. Pada praktikum digunakan killall sleep untuk membersihkan sisa job.
3. Operator > mengarahkan stdout (keluaran normal) ke directories.txt, sedangkan 2> mengarahkan stderr (pesan error) ke errors.txt. Keduanya dipisahkan agar daftar hasil tidak tercampur dengan pesan error seperti permission denied, sehingga jumlah baris setiap file dapat dihitung dan diperiksa dengan jelas.

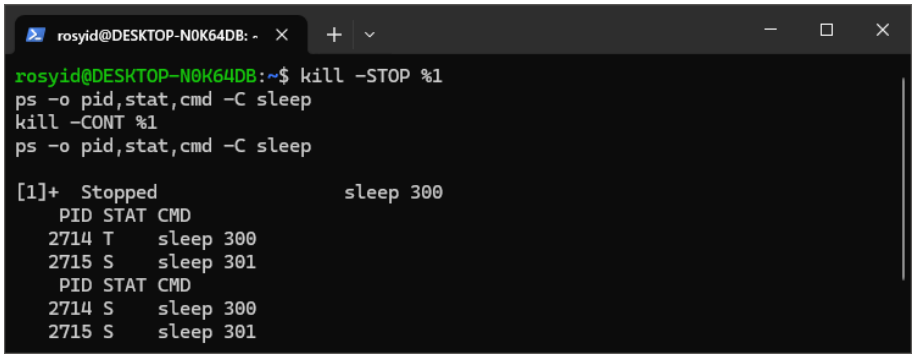
KEGIATAN 3

A. Dokumentasi Kegiatan 3

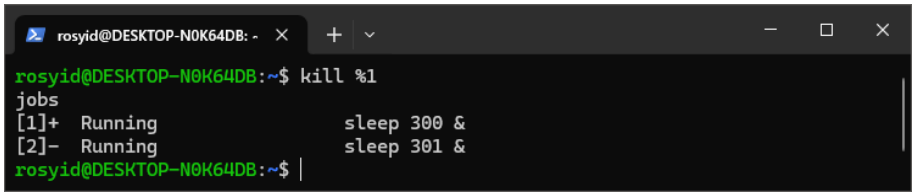
1. Melihat daftar sinyal dengan `kill -l` dan menjalankan dua proses uji



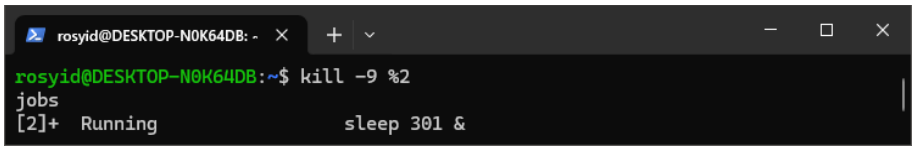
2. Menghentikan sementara dan melanjutkan proses dengan `kill -STOP` dan `kill -CONT`



3. Mengakhiri proses dengan sinyal default (SIGTERM)



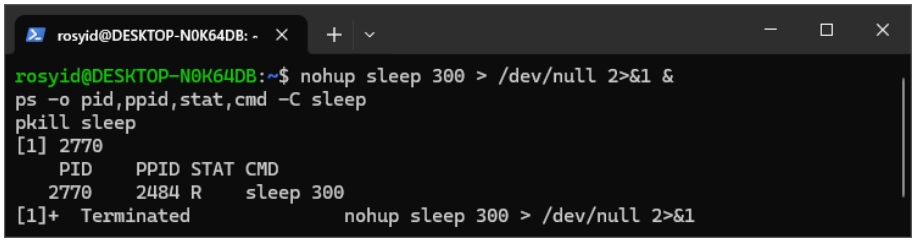
4. Mengakhiri proses dengan `kill -9` (SIGKILL)



5. Menjalankan lima proses `sleep` sekaligus dan mengakhirinya dengan `killall`



6. Menjalankan proses dengan `nohup` lalu mengakhirinya dengan `pkill`



7. Catat hasil pengamatan

Aspek Pengamatan	Output Terminal
STAT proses setelah `kill -STOP`	2714 T sleep 300 2715 S sleep 301
STAT proses setelah `kill -CONT`	2714 S sleep 300 2715 S sleep 301
Pesan `jobs` setelah `kill %1`	[1]+ Running sleep 300 & [2]- Running sleep 301 &
Pesan `jobs` setelah `kill -9 %2`	[2]+ Running sleep 301 &
Hasil `pgrep -c sleep` sebelum dan sesudah `killall`	Sebelum: 5 Sesudah: 0 (muncul lima baris Terminated sleep 60)

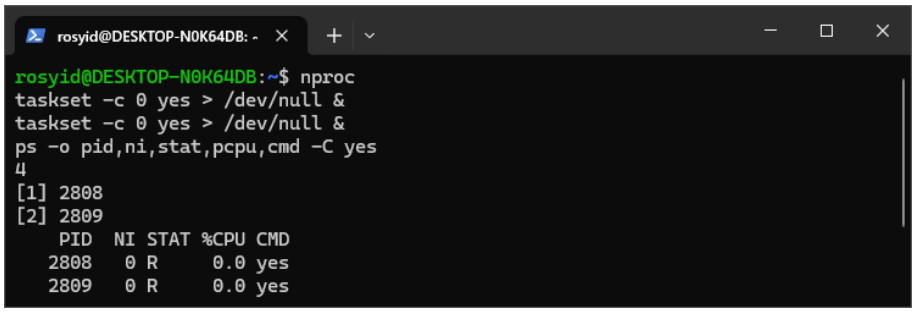
B. Jawaban Analisis Kegiatan 3

1. SIGTERM (15) meminta proses berhenti secara wajar sehingga proses sempat membersihkan sumber daya dan menyimpan data. SIGKILL (9) mematikan proses secara paksa dan tidak dapat ditangkap atau diabaikan, sehingga tidak ada kesempatan untuk cleanup. Karena itu SIGTERM dianjurkan sebagai langkah pertama, dan SIGKILL dipakai hanya bila proses tidak merespons.
2. SIGSTOP tidak dapat ditangkap, diblokir, atau diabaikan oleh proses karena penghentiannya dilakukan oleh kernel. SIGTSTP (Ctrl+Z) dihasilkan oleh terminal dan dapat ditangani oleh program, misalnya diabaikan oleh program tertentu. Pada praktikum, kill -STOP mengubah status proses menjadi T, dan kill -CONT mengembalikannya ke S.
3. nohup membuat proses mengabaikan sinyal SIGHUP sehingga proses tetap berjalan meskipun terminal ditutup. SIGHUP biasanya dikirim ke job shell ketika terminal ditutup. Pada praktikum, output diarahkan ke /dev/null (> /dev/null 2>&1) agar proses tidak menulis ke terminal atau nohup.out.

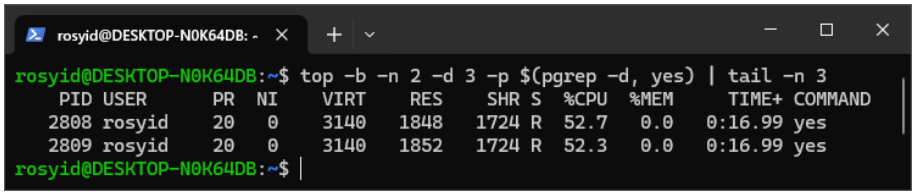
KEGIATAN 4

A. Dokumentasi Kegiatan 4

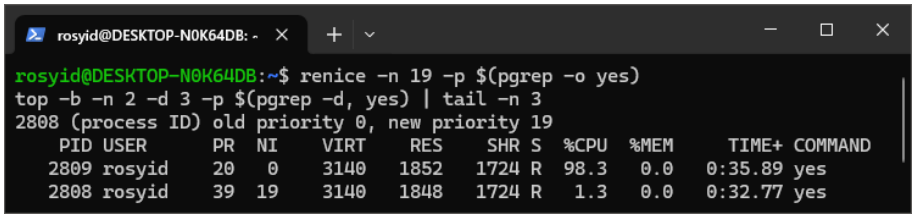
1. Memeriksa jumlah core dengan `nproc` dan menjalankan dua proses `yes` dengan `taskset -c 0`



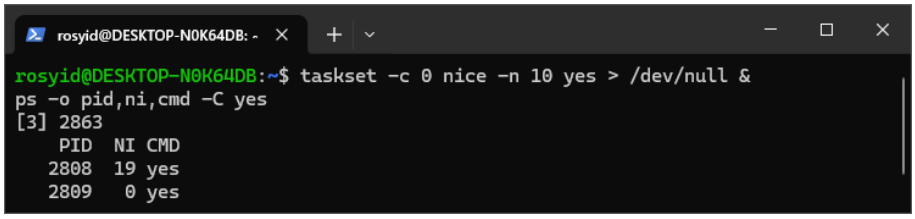
2. Memantau pembagian CPU dengan `top -b`



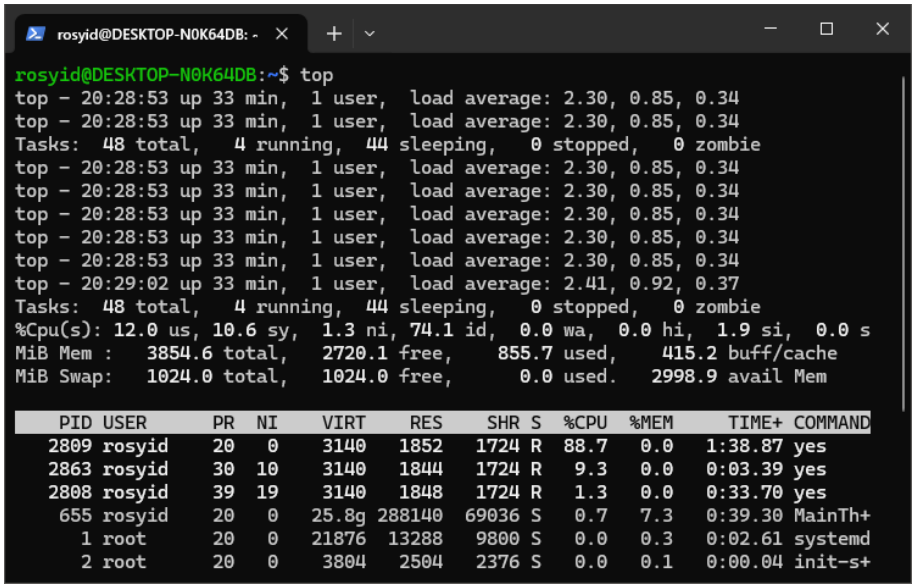
3. Menurunkan prioritas proses pertama dengan `renice -n 19`, lalu memantau kembali



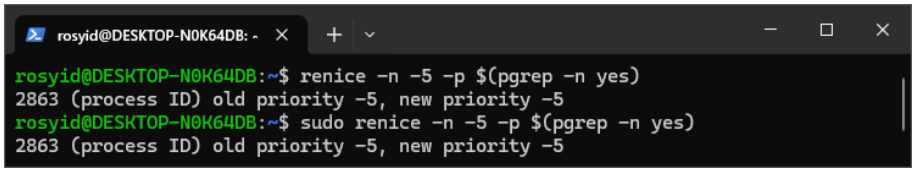
4. Menjalankan proses baru dengan `nice -n 10` dan memeriksa kolom NI



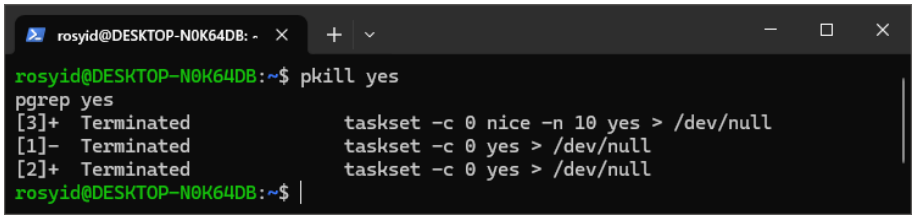
5. Mengubah nilai nice melalui `top` interaktif (tombol `r`)



6. Mencoba nilai nice negatif tanpa dan dengan `sudo`



7. Membersihkan proses uji dengan `pkill yes`



8. Catat hasil pengamatan

Aspek Pengamatan	Output Terminal
NI dan %CPU kedua proses `yes` sebelum `renice`	PID 2808: NI 0, STAT R, %CPU 0.0 PID 2809: NI 0, STAT R, %CPU 0.0 (top: 52.7% dan 52.3%)
NI dan %CPU kedua proses setelah `renice -n 19`	PID 2809: NI 0, %CPU 98.3 PID 2808: NI 19, %CPU 1.3
NI proses yang dijalankan dengan `nice -n 10`	PID 2863: NI 10 (terlihat di top; tidak tercantum pada output ps)
Pesan hasil `renice -n -5` tanpa `sudo`	2863 (process ID) old priority -5, new priority -5

B. Jawaban Analisis Kegiatan 4

1. Kedua proses yes memiliki nilai nice yang sama (0) dan berjalan pada core yang sama karena dipaksa dengan taskset -c 0. Penjadwal CPU Linux (CFS) membagi waktu CPU secara adil antara proses dengan prioritas setara, sehingga keduanya mendapat porsi yang hampir sama (52.7% dan 52.3% pada top).
2. taskset -c 0 memaksa kedua proses berjalan pada core yang sama sehingga keduanya harus bersaing untuk satu core. Jika berjalan pada core berbeda, masing-masing proses memperoleh satu core penuh tanpa persaingan, sehingga nilai nice tidak akan terlihat pengaruhnya pada %CPU.
3. Menurunkan nilai nice berarti menaikkan prioritas proses, dan ini dapat membuat proses lain kekurangan CPU. Karena itu Linux hanya mengizinkan root (atau pengguna dengan hak sudo) untuk menurunkan nilai nice, termasuk ke nilai negatif. User biasa hanya boleh menaikkan nilai nice proses miliknya sendiri.

BAB II
KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa setiap proses di Linux memiliki PID dan PPID dan membentuk hierarki parent–child yang berakar pada PID 1 (systemd). Job control memungkinkan proses dijalankan di foreground, background, dihentikan sementara, lalu dilanjutkan, sedangkan sinyal seperti SIGSTOP, SIGCONT, SIGTERM, dan SIGKILL memberikan kendali atas status proses. Pengaturan nice melalui renice terbukti memengaruhi pembagian CPU saat proses bersaing pada core yang sama, di mana proses dengan nice 19 turun dari 98.3% menjadi 1.3% CPU.